SMART AQUA AI MANAGEMENT

Trần Thanh Tiến Đạt, Trần Đức Việt, Nguyễn Đình Bảo,
Nguyen Đình Đạt, Lê Tự Minh Quang, Trần Huy Hoang,
and Tra Huong Thi Le
FPT University, Danang Campus, Vietnam
{leoken5805@gmail.com, tv462005@gmail.com, dinhbao16888@gmail.com,
nguyendinhdat210920@gmail.com, letuminhquang05@gmail.com,
hoanght152@gmail.com}, and tralth6@fe.edu.vn

Abstract

The "Smart Aqua AI Manager" project is designed to address the challenges of maintaining a stable habitat for aquarium fish. The system utilizes a centralized architecture where an ESP32 microcontroller serves as the primary processing unit, handling both Cloud-synchronized AI inference and IoT connectivity via the Blynk platform. Key features include real-time water quality monitoring (temperature, water level, turbidity), automated care routines (smart feeding, filtration control), and Linear Regression models trained in Python to optimize environmental stability. The system allows users to monitor and control the aquarium remotely through a mobile application, significantly reducing manual maintenance efforts while ensuring a healthy aquatic environment.

I. Introduction

Keeping ornamental fish is a popular hobby but requires knowledge and time to maintain environmental metrics such as temperature, pH, and water cleanliness. In Vietnam and globally, existing smart aquarium solutions are often expensive or lack smart visual monitoring features. The problem is that owners often fail to detect timely signs of illness in fish (lethargy, lying at the bottom) or forget care schedules. This project proposes a low-cost but highly effective IoT and AI integrated solution to automate care and monitor the aquarium 24/7.

II. Related Works and Commercial Comparisons

In the current market, several smart aquarium solutions have emerged, ranging from consumer electronics to high-end crowdfunded projects.

- **Xiaomi Mijia Smart Fish Tank:** Currently the most prominent commercial product. It excels in convenience and aesthetics, offering automated feeding, one-button water changes, and app-based temperature monitoring. However, it lacks advanced AI capabilities for detecting diseases or abnormal fish behavior via computer vision.
- **Bluenero:** A high-end project launched on Indiegogo. It features an integrated HD camera, pH and temperature sensors, and a professional management app. Despite its comprehensive features, the high price point (several hundred USD) makes it less accessible to the general consumer market in Vietnam.

The **Smart Aqua AI Manager** fills this gap by integrating Machine Learning for automated environmental regulation while maintaining a cost-effective hardware architecture while maintaining a cost-effective hardware architecture suitable for local users.

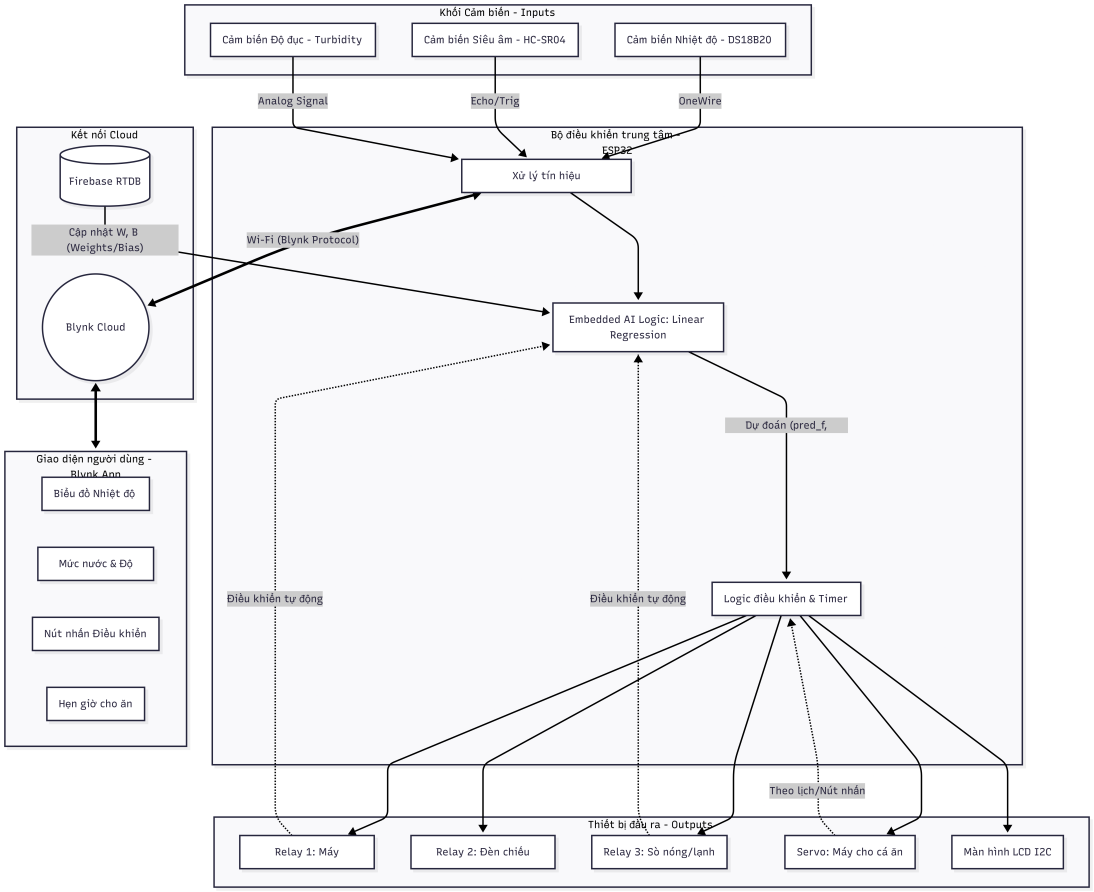


Fig. 1. Block diagram of the developed system.

III. Main proposal

Necessity and topicality, *the scientific and practical* significance of the research topic.

A. System models and block diagram

As illustrated in Fig. 1, the proposed system architecture is designed around the **ESP32 microcontroller**, which functions as the central hub for data acquisition, processing, and communication. Unlike traditional Master-Slave systems, this streamlined design embeds the AI inference logic directly into the microcontroller. Unlike traditional static systems, this design utilizes dynamically updated weights from Firebase to execute its AI logic.

The workflow operates as follows:

1) **Input Layer:** Sensors collect environmental data (Temperature, Water Level, Turbidity).
2) **Processing Layer:** The ESP32 processes raw signals and applies a Linear Regression model with weights synchronized from the cloud to make automated decisions.
3) **Connectivity Layer:** Processed data is synchronized with the **Blynk Cloud**, allowing for real-time visualization on mobile devices.
4) **Output Layer:** The ESP32 controls relays and servos to execute physical tasks like feeding or water filtering.

TABLE I
Requirement Checklist

#	Requirement	checklist	devices
1	Wireless communications: Wifi/Bluetooth/LoRa	x	ESP32 , WiFi
2	Read data from sensors	x	Temp sensor, HC-SR04, Turbidity sensor
3	Process the collected data	x	ESP32
4	Display/Visualize data in display device or web/mobile app	x	Blynk Mobile App, LCD I2C
5	Control the actuators: manually or automatically according to the processed data	x	Relay module, Water pump, Servo Motor
6	AI feature (optional)	x	Python-trained Linear Regression (Cloud-synced weights)

B. Components and peripheral devices

The hardware implementation is divided into sensory inputs and physical actuators:

1) Data Acquisition (Sensors): The system continuously monitors the aquatic environment through three primary sensors connected directly to the ESP32:

- **Temperature Monitoring (DS18B20):** Captures real-time water temperature data to ensure thermal stability.
- **Water Level Detection (HC-SR04):** Measures the distance to the water surface. This data is converted into a percentage to monitor water volume.
- **Water Quality Analysis (Turbidity Sensor):** Measures the cloudiness of the water, providing an analog signal indicating water cleanliness.

2) Actuators: Based on commands from either the Cloud-synced AI Model or the Blynk App, the system controls:

- **Servo Motor:** Rotates to dispense food when triggered by the internal timer or user request.
- **Relay Module:** Controls high-voltage devices including the **Water Pump** (activated when water is turbid) and **Lighting System**.

C. Schematic Circuit

The hardware connections are designed to ensure electrical isolation and stable power distribution. Fig. 2 illustrates the wiring diagram between the ESP32 Central Controller and peripheral modules.

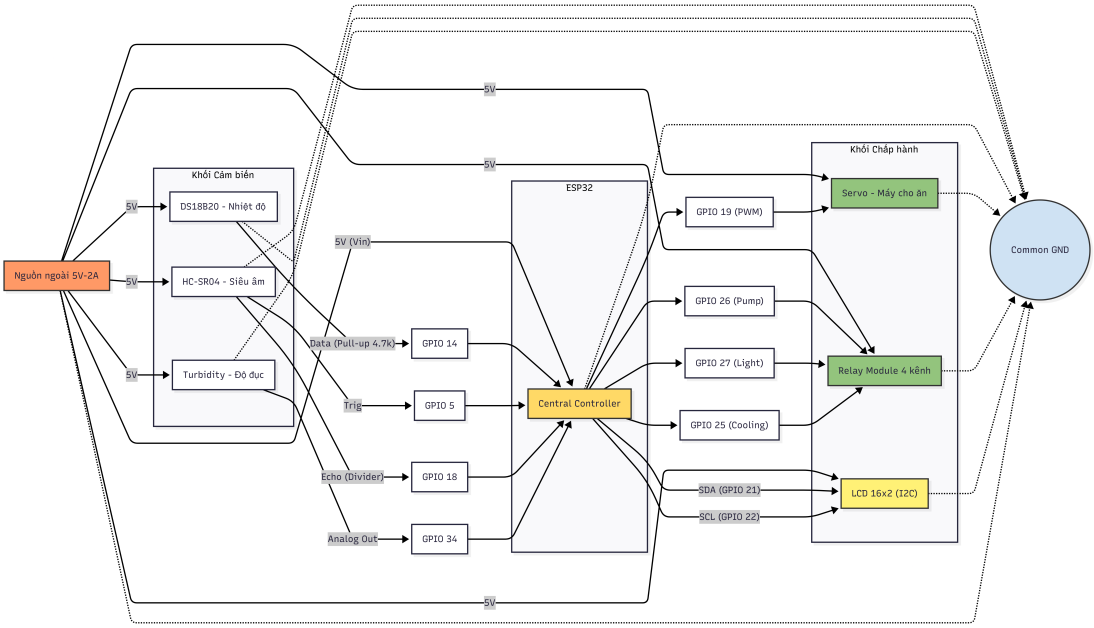


Fig. 2. Wiring schematic of the ESP32 with Sensors and Actuators.

To prevent voltage drops affecting the microcontroller, a separate 5V-2A power supply is used for the Servo motor and Relay coils. The ESP32 GPIO pins operate at 3.3V logic levels, while the Relay module is triggered by Active LOW signals. The detailed pinout mapping is described in Table II.

TABLE II
ESP32 PINOUT MAPPING CONFIGURATION

Component	Pin Name	ESP32 GPIO	Function
DS18B20	VCC	3.3V	Power
	GND	GND	Ground
	DATA	GPIO 14	Temperature Data
HC-SR04	TRIG	GPIO 5	Trigger Pulse
	ECHO	GPIO 18	Receive Pulse
Turbidity Sensor	OUT	GPIO 34	Analog Input
Servo Motor	PWM	GPIO 19	Feeding Mechanism
Relay Module	IN1	GPIO 26	Pump Control
	IN2	GPIO 27	Light Control
	IN3	GPIO 25	Cooling Control
LCD 16x2 (I2C)	VCC	5V	Power
	GND	GND	Ground
	SDA	GPIO 21	I2C Data
	SCL	GPIO 22	I2C Clock

D. Software programming

The software architecture is developed using the Arduino IDE with C++, integrated with the Blynk IoT library and a TinyML inference engine.

1) Hybrid Cloud-Edge AI Control Logic: The system implements a **Linear Regression** model optimized through a hybrid workflow involving offline training and real-time cloud synchronization. This approach ensures high precision while maintaining low computational overhead on the microcontroller.

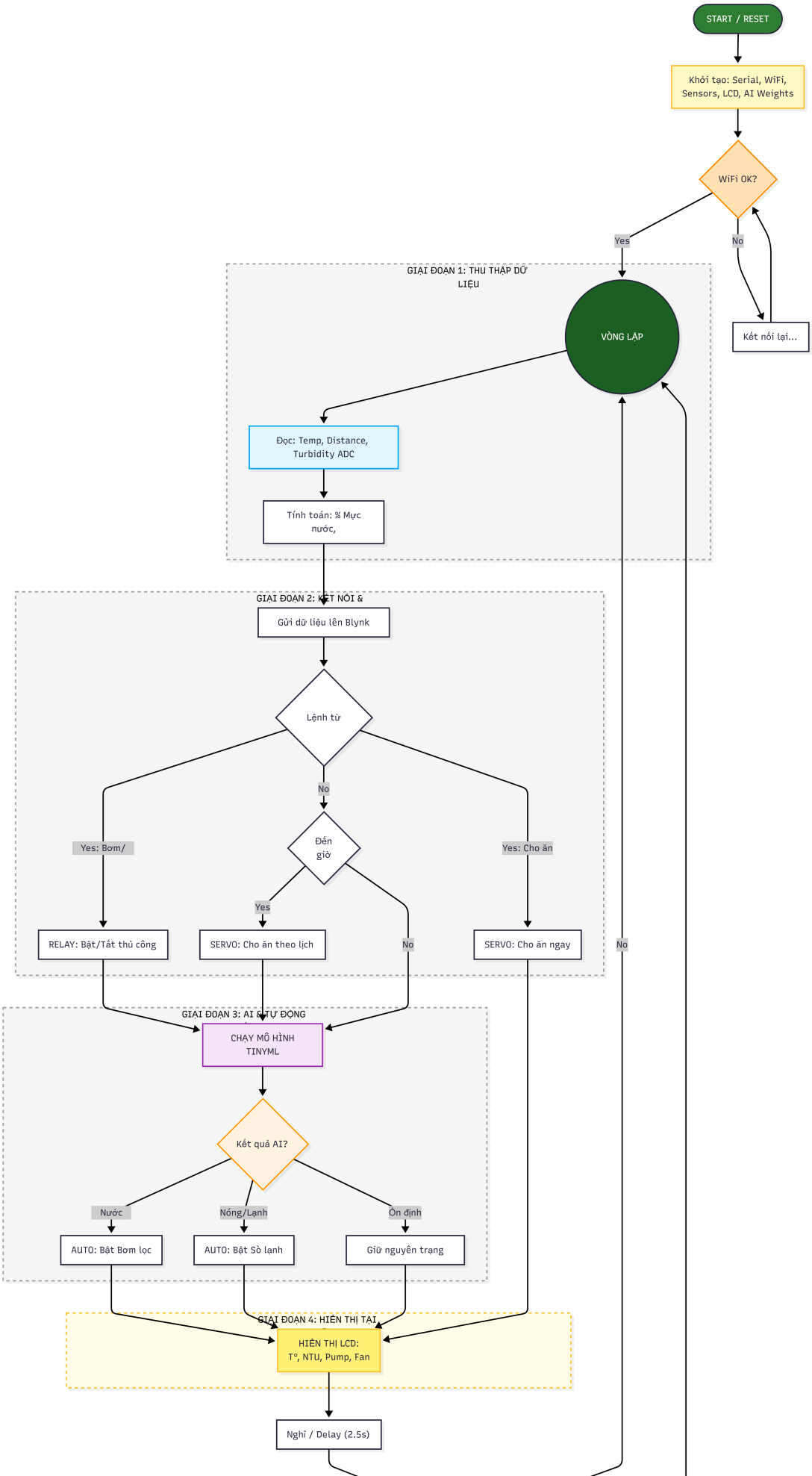
- **Offline Training (Python):** The model is trained externally using Python on a primary dataset. During this phase, the optimal **Weights** (w) and **Bias** (b) are calculated to define the decision boundaries for the actuators.
- **Cloud Parameters Distribution (Firebase):** Instead of hardcoding values into the firmware, the trained parameters (w, b) are stored in the **Firebase Realtime Database**. This allows for remote tuning of the system's sensitivity (e.g., adjusting the cooling threshold) without the need to re-flash the ESP32 code.
- **Edge Inference (ESP32):**
  - **Data Fetching:** The ESP32 periodically synchronizes with Firebase to pull the latest weights into its local memory.
  - **Preprocessing:** Raw sensor data is normalized into a $[0, 1]$ range (e.g., $Temp_{norm} = \frac{T-20}{40-20}$).
  - **Real-time Prediction:** The controller executes the inference locally using the linear equation: $y = w \cdot x + b$.
  - **Actuation:** Based on the predicted value (y), the system triggers the **Relay Module** for filtration (Pump) and temperature regulation (Cooling) instantaneously.

2) Blynk App Interface: The mobile application provides bidirectional control:

- **Monitoring:** Visualizes temperature charts, water level gauges, and turbidity values.
- **Control:** Allows users to trigger "Feed Now", set feeding schedules, and manually toggle pumps/lights.

E. Programming Flowchart

Fig. **??** illustrates the program flow, starting from sensor initialization, Wifi connection, followed by the loop of reading sensors, running AI inference, handling Blynk requests, and updating actuators.


START / RESET
Khởi tạo: Serial, WiFi, Sensors, LCD, AI Weights
WiFi OK?
Yes
No
Kết nối lại...
GIAI ĐOẠN 1: THU THẬP DỮ LIỆU
VÒNG LẶP
Đọc: Temp, Distance, Turbidity ADC
Tính toán: % Mực nước,
GIAI ĐOẠN 2: KẾT NỐI &
Gửi dữ liệu lên Blynk
Lệnh từ
No
Yes: Bơm/
Đến giờ
Yes: Cho ăn
Yes
RELAY: Bật/Tắt thủ công
SERVO: Cho ăn theo lịch
No
SERVO: Cho ăn ngay
No
GIAI ĐOẠN 3: AI & TỰ ĐỘNG
CHẠY MÔ HÌNH TINYML
Kết quả AI?
Nước
Nóng/Lạnh
Ổn định
AUTO: Bật Bơm lọc
AUTO: Bật Sò lạnh
Giữ nguyên trạng
GIAI ĐOẠN 4: HIỂN THỊ TẠI
HIỂN THỊ LCD: T°, NTU, Pump, Fan
Nghỉ / Delay (2.5s)

IV. Results and discussion

A clear concise description of the research methods used.

A. Prototype Implementation

The hardware was assembled on a PCB/Breadboard integrating the ESP32, relays, and sensors. The system was installed in a 50-liter test aquarium. Cite here [1]

B. Experimental Results

The system successfully maintained connection with the Blynk server over a 24-hour test period. The Linear Regression model correctly identified high turbidity levels and activated the pump automatically using the weights fetched from Firebase. [2]

C. Discussion

The integration of ESP32 proved effective for handling both IoT communication and real-time AI inference. The hybrid approach of training on Python and deploying on ESP32 proved to be highly flexible for environmental management. However, future improvements could include a camera module for more advanced visual fish disease detection.

V. Conclusion

The "Smart Aqua AI Manager" successfully demonstrates a low-cost, automated solution for aquarium care. By combining ESP32, TinyML, and Blynk, the system reduces the workload for fish owners while ensuring optimal living conditions for the fish. [3]

VI. Author's contribution

TABLE III
Author's contribution

#	Student ID	Student Name	Tasks	Contribution
1	DE190478	Lê Tự Minh Quang	Draw block diagram, flowchart	20%
2	DE190953	Trần Đức Việt	Study conception and design	25%
3	DE191024	Trần Thanh Tiến Đạt	Program ESP32	25%
4	DE190927	Nguyễn Đình Đạt	Write report, prepare Presentation	10%
5	DE190972	Trần Huy Hoàng	Write report, prepare Presentation	10%
6	DE190614	Nguyễn Đình Bảo	Write report, prepare Presentation	10%
Total				100%

References

[1] M. Tellez, S. El-Tawab, and H. M. Heydari, "Improving the security of wireless sensor networks in an iot environmental monitoring system," in *2016 IEEE systems and information engineering design symposium (SIEDS)*. IEEE, 2016, pp. 72–77.

[2] X. Yao, J. Wang, M. Shen, H. Kong, and H. Ning, "An improved clustering algorithm and its application in iot data analysis," *Computer Networks*, vol. 159, pp. 63–72, 2019.

[3] D. Li, D. Zhao, Y. Chen, and Q. Zhang, "Deepsign: Deep learning based traffic sign recognition," in *2018 international joint conference on neural networks (IJCNN)*. IEEE, 2018, pp. 1–6.